Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «КиСБ»
Тема: «Избыточное кодирование данных в информационных системах. Код Хемминга. Итеративные коды»

Выполнил:
Студент 4-го курса
Группы АС-65
Лопато А.В.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы**:** приобретение практических навыков кодирования/де- кодирования двоичных данных при использовании кода Хемминга. приобретение практических навыков кодирования/дс- кодирования двоичных данных при использовании итеративных кодов .

## Ход работы

## Вариант 13

Часть 1

На основе информационного сообщения, представленного символами русского/английского алфавитов, служебными символа- ми и цифрами, содержащегося в некоторт текстовом файле (согла- 42 Лабораторная работа N2 4 совать с преподавателем), сформировать информационнос сообще- ние в двоичном виде; длина сообщения в бинарном виде должна быть не менее 16 символов. Для выполнения этого задания можно ис- пользовать коды ASCIl символов алфавита либо результаты лабора торной —ты N2 3. 2. Для полученного информационного слова прове- рочную матрицу Хемминга (значение минимального кодового расстояния согласовать с преподавателем). З . Используя построенную матрицу, вычислить избыточные символы (слово Х). 4. Принять слово следующим числом ошибок: 0, 1, 2. Позт,щя ошибки определяется (генерируется) случайным образом. 5. Для полученного слова Уп = Ук, Yr, используя уже известную проверочную матрицу Хемминга, вновь вычислить избыточные символы (обозначим их Г), используя выражение (4.6). б. Вычислить и проанализировать синдром. В случае, если анализ синдрома показал, что было передано с ошибкой (или 2 ошибками), унарный вектор ошибки Е,, = щ, ф, и исправить одиночную ошибку, используя формулу (4.7); проанализировать ситуацию при возник- новении ошибки в 2 битах. 7. Результаты формить в виде шчега по установленным правилам. Файл анализируемой информации (а скмjтветственно и интер- фейс приложения) должен содержать исходное информационное сообщение, значения величин К, r, п, проверочную матрицу Хем- минга слово Х, Х, Yn,Yr, Yr', синдром S, вектор ошибки Еп. Программа не должна быть чувствительна к длине информацион- ного сообщения.

Часть 2

Разработать собственное приложение, которое позволяет вы- полнять следующие операции: 1) вписывать произвольное двоичное представление инфор мационного слова Хк (кодируемой информации) длиной К битов в двумерную матрицу размерностью в соответствии с вариантом либо в трехмерную матрицу в соответствии с вариантом (указаны в табл. 5.2); 2) вычислять проверочные биты (биты паритетов): а) по двум; б) по трем; в) по четырем направлениям (группам паритетов); З) формировать кодовое слово Х, присоединением избыточ- ных символов к информационному слову; 4) генерировать ошибку произвольной кратности (i, i > О), распределенную случайным (фазом среди символов слова Х, в результате чего формируется кодовое слово Уп; 5) определять местоположение ошибочных символов итера- тивным кодом в слове Уп в соответствии с исполызусмыми груп- пами паритетов по пункту (2) и исправлять ошибочные символы (результат исправления слово Уп '); б) выполнять анализ корректирующей способности исполь- зуемого кода (количественная оценка) путем сравнения соответст- вующих слов Х, и У,,' ; результат анализа может быть представлен в виде отношения общего числа сгенерированных кодовых слов с ошибками

определенной одинаковой кратности (с одной ошибкой, с двумя ошибками и т. д.) к числу кодовых слов, содержащих ИзбыточнсЕ данных в инфрмационных системах. Итеративные коды 55 ошибки этой кратности, которые правильно обнаружены и которые правильно скорректированы.

Lab2part1:

```
import random

def text_to_bits(text):

    encoded = text.encode('cp1251', errors='ignore')
    bits = []
    for byte in encoded:
        bits.extend([int(b) for b in format(byte, '08b')])
    return bits

def calculate_r(k):

    r = 2
    while (2**r) < (k + r + 1):
        r += 1
    return r

def generate_hamming_matrix(k, r):

    n = k + r

    I_cols = [[1 if i == j else 0 for j in range(r)] for i in range(r)]

    P_cols = []
    for i in range(1, 2**r):
        col = [int(x) for x in format(i, f'0{r}b')]
        if sum(col) >= 2:
            P_cols.append(col)
            if len(P_cols) == k:
                break

    if len(P_cols) < k:
        raise ValueError("Не хватает комбинаций для матрицы. Увеличьте r.")

    H_cols = P_cols + I_cols
    return H_cols, P_cols

def print_matrix(H_cols, r, n):
    for row_idx in range(r):
        row = [H_cols[col_idx][row_idx] for col_idx in range(n)]
        print("  [" + " ".join(map(str, row)) + "]")

def simulate_transmission(X, num_errors):
    Y = X.copy()
    n = len(Y)
    error_positions = random.sample(range(n), num_errors)
    for pos in error_positions:
        Y[pos] ^= 1
    return Y, error_positions

def main():
    text_message = "Кот"
    Y_k = text_to_bits(text_message)
    k = len(Y_k)

    r = calculate_r(k)
```

```
    n = k + r

    print("=== ЭТАП 1: Параметры и генерация ===")
    print(f"Исходное сообщение: '{text_message}'")
    print(f"Информационное слово (Y_k): {''.join(map(str, Y_k))}")
    print(f"Параметры: K = {k}, r = {r}, n = {n}")

    H_cols, P_cols = generate_hamming_matrix(k, r)
    print("\nПроверочная матрица Хемминга H (r x n):")
    print_matrix(H_cols, r, n)

    Y_r = []
    for i in range(r):

        bit = sum(Y_k[j] * P_cols[j][i] for j in range(k)) % 2
        Y_r.append(bit)

    X = Y_k + Y_r
    print(f"\nИзбыточные символы (Y_r): {''.join(map(str, Y_r))}")
    print(f"Сформированное кодовое слово (X): {''.join(map(str, X))}")

    for num_errors in [0, 1, 2]:
        print(f"\n{'='*40}")
        print(f"=== ТЕСТ: Передача с {num_errors} ошибками ===")

        Y_n, true_err_pos = simulate_transmission(X, num_errors)
        Y_k_recv = Y_n[:k]
        Y_r_recv = Y_n[k:]

        print(f"Принятое слово (Y_n): {''.join(map(str, Y_n))}")
        print(f"Принятые избыточные (Y_r): {''.join(map(str, Y_r_recv))}")
        if num_errors > 0:
            print(f"Фактические позиции ошибок (индексы): {true_err_pos}")

        Y_r_prime = []
        for i in range(r):
            bit = sum(Y_k_recv[j] * P_cols[j][i] for j in range(k)) % 2
            Y_r_prime.append(bit)
        print(f"Вычисленные избыточные (Y_r'): {''.join(map(str,
Y_r_prime))}")

        S = [(Y_r_recv[i] ^ Y_r_prime[i]) for i in range(r)]
        print(f"Синдром (S): {''.join(map(str, S))}")

        E_n = [0] * n
        if sum(S) == 0:
            print("Синдром равен 0 Ошибок нет")
            print(f"Вектор ошибки (E_n): {''.join(map(str, E_n))}")
        else:
            print("Синдром не равен 0 ошибка")

            if S in H_cols:
                error_idx = H_cols.index(S)
                E_n[error_idx] = 1
                print(f"Вектор ошибки (E_n): {''.join(map(str, E_n))}")

                corrected_Y_n = [(Y_n[i] ^ E_n[i]) for i in range(n)]
                print(f"Исправленное слово:  {''.join(map(str,
corrected_Y_n))}")

                if num_errors == 1:
                    print("Одиночная ошибка исправлена")
                elif num_errors == 2:
                    print("Было 2 ошибки")
```

```
                else:

                    print("Синдром не найден")

if __name__ == "__main__":
    main()
```

Результат:

```
=== ЭТАП 1: Параметры и генерация ===
Исходное сообщение: 'Кот'
Информационное слово (Y_k): 110010101110111011110010
Параметры: K = 24, r = 5, n = 29

Проверочная матрица Хемминга H (r x n):
  [0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0]
  [0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 1 1 1 1 1 1 0 1 0 0 0]
  [0 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 0 1 1 0 0 1 0 0]
  [1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 0 1 1 0 0 1 1 0 0 0 0 0 1 0]
  [1 1 0 1 1 0 1 0 1 0 1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 0 0 0 1]

Избыточные символы (Y_r): 00001
Сформированное кодовое слово (X): 110010101110111011111001000001

========================================
=== ТЕСТ: Передача с 0 ошибками ===
Принятое слово (Y_n): 110010101110111011111001000001
Принятые избыточные (Y_r): 00001
Вычисленные избыточные (Y_r'): 00001
Синдром (S): 00000
Синдром равен 0 Ошибок нет
Вектор ошибки (E_n): 00000000000000000000000000000
```

```
========================================
=== ТЕСТ: Передача с 1 ошибками ===
Принятое слово (Y_n): 111010101110111011111001000001
Принятые избыточные (Y_r): 00001
Фактические позиции ошибок (индексы): [2]
Вычисленные избыточные (Y_r'): 00111
Синдром (S): 00110
Синдром не равен 0 ошибка
Вектор ошибки (E_n): 00100000000000000000000000000
Исправленное слово:  110010101110111011111001000001
Одиночная ошибка исправлена

========================================
=== ТЕСТ: Передача с 2 ошибками ===
Принятое слово (Y_n): 100010101110111001111001000001
Принятые избыточные (Y_r): 00001
Фактические позиции ошибок (индексы): [1, 16]
Вычисленные избыточные (Y_r'): 10010
Синдром (S): 10011
Синдром не равен 0 ошибка
Вектор ошибки (E_n): 00000000000001000000000000000
Исправленное слово:  100010101110101001111001000001
Было 2 ошибки
```

lab2part2:

```
import random


class IterativeCode:
    def __init__(self, k1=4, k2=6):
        self.k1 = k1
        self.k2 = k2
        self.k = k1 * k2  # 24 бита
        self.num_diags = self.k1 + self.k2 - 1

    def _get_diag_index(self, r, c):
        return r - c + (self.k2 - 1)

    def generate_random_word(self):
        return [random.choice([0, 1]) for _ in range(self.k)]

    def encode(self, data, num_groups):

        assert len(data) == self.k
        matrix = [data[i * self.k2:(i + 1) * self.k2] for i in
range(self.k1)]

        p1_row = [sum(row) % 2 for row in matrix]

        p2_col = [sum(matrix[r][c] for r in range(self.k1)) % 2 for c in
```

```
range(self.k2)]

        encoded = data + p1_row + p2_col

        if num_groups == 3:
            p3_diag = [0] * self.num_diags
            for r in range(self.k1):
                for c in range(self.k2):
                    d_idx = self._get_diag_index(r, c)
                    p3_diag[d_idx] ^= matrix[r][c]
            encoded += p3_diag

        return encoded, matrix

    def apply_errors(self, codeword, num_errors):
        corrupted = codeword.copy()
        n = len(corrupted)
        error_positions = random.sample(range(n), num_errors)
        for pos in error_positions:
            corrupted[pos] ^= 1
        return corrupted, error_positions

    def decode(self, received, num_groups):
        recv_data = received[:self.k]
        recv_p1 = received[self.k: self.k + self.k1]
        recv_p2 = received[self.k + self.k1: self.k + self.k1 + self.k2]

        if num_groups == 3:
            recv_p3 = received[self.k + self.k1 + self.k2:]

        matrix = [recv_data[i * self.k2:(i + 1) * self.k2] for i in
range(self.k1)]

        max_iterations = 10
        for iteration in range(max_iterations):
            cur_p1 = [sum(row) % 2 for row in matrix]
            cur_p2 = [sum(matrix[r][c] for r in range(self.k1)) % 2 for c in
range(self.k2)]

            s1 = [recv_p1[i] ^ cur_p1[i] for i in range(self.k1)]
            s2 = [recv_p2[i] ^ cur_p2[i] for i in range(self.k2)]

            s3 = [0] * self.num_diags
            if num_groups == 3:
                cur_p3 = [0] * self.num_diags
                for r in range(self.k1):
                    for c in range(self.k2):
                        d_idx = self._get_diag_index(r, c)
                        cur_p3[d_idx] ^= matrix[r][c]
                s3 = [recv_p3[i] ^ cur_p3[i] for i in range(self.num_diags)]

            total_syndromes = sum(s1) + sum(s2) + sum(s3)
            if total_syndromes == 0:
                break

            max_fails = 0
            best_bit = None

            for r in range(self.k1):
                for c in range(self.k2):
                    fails = s1[r] + s2[c]
                    if num_groups == 3:
                        d_idx = self._get_diag_index(r, c)
                        fails += s3[d_idx]
```

```
                    if fails > max_fails:
                        max_fails = fails
                        best_bit = (r, c)

            if best_bit and max_fails >= 2:
                r, c = best_bit
                matrix[r][c] ^= 1
            else:
                break

        corrected_data = []
        for row in matrix:
            corrected_data.extend(row)

        return corrected_data, total_syndromes > 0

    def analyze(self, max_errors=4, trials=1000):
        print(f"\n--- Анализ корректирующей способности (k={self.k},
испытаний={trials}) ---")

        for groups in [2, 3]:
            print(f"\nГруппы паритетов: {groups} " + (
                "(Строки, Столбцы)" if groups == 2 else "(Строки, Столбцы,
Диагонали)"))
            print(f"{'Кратность ошибки':<20} | {'Обнаружено (%)':<15} |
{'Скорректировано (%)'}")
            print("-" * 60)

            for num_errors in range(1, max_errors + 1):
                detected_count = 0
                corrected_count = 0

                for _ in range(trials):
                    data = self.generate_random_word()
                    encoded, _ = self.encode(data, groups)

                    corrupted, _ = self.apply_errors(encoded, num_errors)

                    decoded_data, has_syndromes = self.decode(corrupted,
groups)

                    if has_syndromes or decoded_data != data:
                        detected_count += 1

                    if decoded_data == data:
                        corrected_count += 1

                det_percent = (detected_count / trials) * 100
                corr_percent = (corrected_count / trials) * 100
                print(f"{num_errors:<20} | {det_percent:<15.1f} |
{corr_percent:.1f}")


def main():
    coder = IterativeCode(k1=4, k2=6)
    data = coder.generate_random_word()

    print("=== Демонстрация работы (2 группы паритетов) ===")
    print(f"1) Информационное слово (X_k): {''.join(map(str, data))}")

    encoded, matrix = coder.encode(data, num_groups=2)
    print("   Матрица 4x6:")
    for row in matrix:
```

```
            print("   " + str(row))

        print(f"3) Кодовое слово (X_n): {''.join(map(str, encoded))}")


        corrupted, err_pos = coder.apply_errors(encoded, num_errors=1)
        print(f"4) Слово с ошибкой (Y_n) (позиции {err_pos}): {''.join(map(str,
    corrupted))}")


        corrected, _ = coder.decode(corrupted, num_groups=2)
        print(f"5) Исправленное слово (Y_n'): {''.join(map(str, corrected))}")
        print(f"   Успешно исправлено: {corrected == data}")


        coder.analyze(max_errors=4, trials=1000)


    if __name__ == "__main__":
        main()
```

Результат:

```
=== Демонстрация работы (2 группы паритетов) ===
1) Информационное слово (X_k): 010101010000100110010110
   Матрица 4x6:
   [0, 1, 0, 1, 0, 1]
   [0, 1, 0, 0, 0, 0]
   [1, 0, 0, 1, 1, 0]
   [0, 1, 0, 1, 1, 0]
3) Кодовое слово (X_n): 010101010000100110010110111111110101
4) Слово с ошибкой (Y_n) (позиции [27]): 010101010000100110010110111011010 1
5) Исправленное слово (Y_n'): 010101010000100110010110
   Успешно исправлено: True

--- Анализ корректирующей способности (k=24, испытаний=1000) ---

Группы паритетов: 2 (Строки, Столбцы)
Кратность ошибки     | Обнаружено (%)  | Скорректировано (%)
------------------------------------------------------------
1                    | 31.8            | 100.0
2                    | 85.2            | 35.3
3                    | 99.0            | 6.7
4                    | 100.0           | 0.3

Группы паритетов: 3 (Строки, Столбцы, Диагонали)
Кратность ошибки     | Обнаружено (%)  | Скорректировано (%)
------------------------------------------------------------
1                    | 47.9            | 100.0
2                    | 75.9            | 86.1
3                    | 91.7            | 58.9
4                    | 98.9            | 27.5
```

**Вывод:** приобрел практических навыков кодирования/де- кодирования двоичных данных при использовании кода Хемминга. приобретел практических навыков кодирования/дс-кодирования двоичных данных при использовании итеративных кодов .